

C950 Task-1 WGUPS Algorithm Overview

C950 Task-1 WGUPS Algorithm Overview

(Task-1: The planning phase of the WGUPS Routing Program)

Cole Linke

ID: 011917160

WGU Email: clink24@wgu.edu

07/10/25

C950 Data Structures and Algorithms II

Introduction

The task is to determine an algorithm to efficiently route and deliver 40 packages for Western Governors University Parcel Service (WGUPS) using two drivers and three trucks. The program must also include an intuitive interface for the user to view the delivery status of any package at any given time.

Assumptions:

- Each truck can carry a maximum of 16 packages, and the ID number of each package is unique.
- The trucks travel at an average speed of 18 miles per hour and have an infinite amount of gas with no need to stop.
- There are no collisions.
- Three trucks and two drivers are available for deliveries. Each driver stays with the same truck as long as that truck is in service.
- Drivers leave the hub no earlier than 8:00 a.m., with the truck loaded, and can return to the hub for packages if needed.
- The delivery and loading times are instantaneous (i.e., no time passes while at a delivery or when moving packages to a truck at the hub). This time is factored into the calculation of the average speed of the trucks.
- There is up to one special note associated with a package.
- The delivery address for package #9, Third District Juvenile Court, is wrong and will be corrected at 10:20 a.m. WGUPS is aware that the

address is incorrect and will be updated at 10:20 a.m. However, WGUPS does not know the correct address (410 S. State St., Salt Lake City, UT 84111) until 10:20 a.m.

- The distances provided in the “WGUPS Distance Table” are equal regardless of the direction traveled.

The day ends when all packages are delivered and all trucks are back at the hub.

A. Algorithm Identification

For my solution, I used a combination of the greedy and nearest-neighbor algorithms. The algorithm demonstrates nearest-neighbor behavior by always selecting the next delivery location closest to the truck's current location. The algorithm is greedy because it only accounts for the shortest immediate distance at each stop and does not look ahead or optimize the entire route. The algorithm is self-adjusting because the truck's location changes after every delivery, meaning that the algorithm continues to re-evaluate the remaining options and respond dynamically.

- At the hub, a truck is loaded with packages
- The algorithm will check through all the packages' destination addresses and choose the one that is closest relative to the truck's starting location at the hub (nearest-neighbor)
- The truck will travel to that location and deliver the package, removing it from its load, and repeat the previous step, only testing its current location (greedy)

- This process continues until all packages are delivered

B. Data Structure Identification

I used a hash table as the self-adjusting data structure to store all the package information. Each entry contains a package object with relevant delivery details like package ID, address, deadline, weight, status, and timestamps. Hash tables allow for fast lookup, insert, and update functions. The hash table is self-adjusting because it resizes automatically when the number of stored packages exceeds its initial capacity.

B1. Explanation of Data Structure

The hash table stores each package object using the package ID as its key. When calculating the index of a package, the ID is hashed. If multiple packages are mapped to the same index (collision), the hash table will use chaining to store them in a list at that index, so no data is lost.

C1. Algorithm's Logic

The core algorithm finds the most efficient route for each truck using a nearest-neighbor and greedy approach. It loops through a process of selecting the next closest address based on the truck's current location.

C950 Task-1 WGUPS Algorithm Overview

```
1  BEGIN
2      FOR each truck
3
4          INITIALIZE empty list of undelivered packages
5
6          FOR each package ID in truck's packages
7              SEARCH for package in hash_table using package ID
8              IF package exists
9                  APPEND package to undelivered packages
10             ELSE
11                 PRINT error message
12
13         CLEAR truck's package list
14
15         SET current time to truck's depart time
16
17         WHILE there are undelivered packages
18             INITIALIZE shortest distance to infinity
19             INITIALIZE next package to none
20
21             FOR each package in undelivered packages
22                 CALCULATE distance from current truck address to package address
23
24                 IF this distance is less than shortest distance
25                     UPDATE shortest distance
26                     UPDATE next package to that package
27
28             deliver next package:
29                 APPEND next package's ID to truck's package list
30                 ADD shortest distance to truck's total mileage
31                 CALCULATE travel time using truck's speed
32                 ADD travel time to current time
33                 SET package's delivery time and status
34                 UPDATE truck's location to next package's address
35                 REMOVE next package from undelivered packages
36
37         after all deliveries:
38             CALCULATE truck's return distance to hub
39             ADD return distance to truck's total mileage
40             UPDATE truck's return time
41             UPDATE truck's location to hub
42     END
```

C2. Development Environment

Processor: 3.8 GHz AMD Ryzen 7 9700X

Memory: 32 GB

Operating System: Windows 11 Home

Interpreter: Python 3.13

IDE: PyCharm 2025.1.1.1

C3. Space and Time complexity using Big-O notation

HASH TABLE.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
<code>__init__()</code>	$O(n)$	$O(n)$	n = initial capacity
<code>insert()</code> <code>search()</code> <code>remove()</code>	$O(n)$	$O(1)$	n = items in bucket
<code>print table()</code> <code>get all items()</code>	$O(n)$	$O(n)$	n = key-value pairs in bucket
<code>__str__()</code> <code>__repr__()</code>	$O(1)$	$O(1)$	constant

CSV IMPORT.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
<code>load files()</code> <code>load packages()</code>	$O(n)$	$O(n)$	n = rows in CSV files

DELIVERY.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
load address() return to hub()	$O(n)$	$O(1)$	n = number of addresses
distance between()	$O(1)$	$O(1)$	constant
delivery()	$O(n^2)$	$O(n)$	n = number of packages per truck

PACKAGE.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
__init__() __repr__() __str__() delivery update() address update() delay check()	$O(1)$	$O(1)$	constant

TRUCK.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
__init__() __repr__() __str__()	$O(1)$	$O(1)$	constant

UI.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
ui while loop()	$O(n \log n)$	$O(n)$	n = number of packages in hash table
display all()	$O(n \log n)$	$O(n)$	n = number of packages
look up package()	$O(n)$	$O(1)$	n = number of items in bucket
time-specific display all()	$O(n \log n)$	$O(n)$	n = number of packages
time-specific look up package()	$O(n)$	$O(1)$	n = number of items in bucket

UI.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
address / status update()	$O(n)$	$O(1)$	n = number of packages

MAIN.PY	TIME COMPLEXITY	SPACE COMPLEXITY	NOTES
run delivery()	$O(n^2)$	$O(n)$	n = number of packages per truck
run ui()	$O(n)$	$O(n)$	n = number of packages

Overall, the program has a worst-case time complexity of $O(n^2)$, where n is the number of packages per truck. This is due to the delivery function using nested loops to find the nearest neighbor of the remaining packages at each step. Additionally, the overall space complexity is $O(n)$, where n is the total number of packages. This is because each package is stored in a hash table and processed individually.

C4. Scalability and Adaptability

The solution is capable of scaling because it can adequately accommodate a growing number of packages. First, the hash table resizes automatically when the number of packages exceeds its initial capacity. It also maintains fast lookup, insert, and update times as the dataset grows. Second, the nearest neighbor algorithm assesses the next closest delivery, keeping its core functionality regardless of how many packages are added. The program is also flexible enough to support future changes, such as adding more trucks or trucks with different speeds and load sizes. However, due to the program's $O(n^2)$ time complexity, the current iteration is not optimized

for extremely large datasets and may experience performance issues as the number of packages increases.

C5. Software Efficiency and Maintainability

The program is efficient because it runs in a required worst-case time complexity of $O(n^2)$. All other functions run in either $O(1)$ or $O(n)$ time. The hash table's insert, search, and update functions will also run in $O(1)$ or $O(n)$, depending on whether or not there are multiple items in a bucket.

The program is maintainable because it is written in a modular structure. Each component is separated into its own file, which allows other developers to find and modify portions of code easily. The program is also heavily documented and uses descriptive function and variable names.

C6. Self-Adjusting Data Structures

The primary strength of the hash table is its efficiency. As long as there is only one item per bucket, the hash table will return an $O(1)$ time complexity for search, insert, and update functions. Another strength is its self-adjusting behavior; its ability to automatically resize and use chaining to handle collisions contributes to its scalability.

A weakness of the hash table is that if multiple items are stored in the same bucket, the time complexity can increase to $O(n)$. The hash table also does not have any order to the data inserted. In my solution, to better display the output to the user, I used Python's default Timsort algorithm to sort my data, which adds an additional $O(n \log n)$ time complexity.

C7. Data Key

I chose package ID as my data key. I chose this because each package has a unique key. All other components (address, deadline, weight, zip code, etc.) could appear in multiple packages, so they aren't guaranteed to be unique. If more packages are added to the data set, they might have duplicate values, but the ID will always be unique, ensuring there won't be a conflict when inserting, updating, or searching.

D. Sources

My task 2 code

E. Professional Communication

Nothing to write. Run your document through <https://www.grammarly.com/>